

My role

1a. At the wk12 kickoff we split responsibilities by module: Harish took hardware and communications, Zian took path planning, Robin took state machines, and Edward and I shared computer vision—I delegated the model training to Edward so I could focus on the detection-to-GPS pipeline. That was the explicit allocation. Table 1 shows where reality diverged.

Table 1: Assigned vs. actual responsibilities

Member	Assigned (wk12)	Actual (by wk22)
Apollo	CV pipeline (shared w/ Edward)	CV pipeline, simulation framework, ground station, safety layer, progressive test ladder, documentation, complaint letter, report coordination
Robin	State machine / flight execution	State machine, integration with CV output, flight-day piloting coordination
Harish	Hardware setup, comms between components	Hardware wiring, Cube-Pi serial link, [APOLLO: confirm what else Harish delivered]
Edward	CV model training (delegated by Apollo)	Trained a custom model; we didn't use it—went with standard YOLO person detection instead. [APOLLO: did Edward do anything else?]
Zian	Path planning	Path planning module. [APOLLO: confirm scope—did Zian's planner make it into the final system?]

The pattern is visible in the right-hand column: my row kept growing while others stayed roughly static. Every pickup after wk12 was triggered by a gap nobody else was filling—camera calibration because nobody else had the camera, simulation because three cancelled flight days left us with no hardware access, the ground station because the Pi needed headless browser-based control. I initially framed this drift as diligence. I now read it differently. Drawing on Lencioni's (2002) five-dysfunctions model from our Professional Practice sessions, solo building was partly an *absence-of-trust* pattern: I didn't trust the team to navigate work I cared about, so I ring-fenced it. My Ukrainian engineering instinct says one person delivers; this project showed me that instinct is a failure mode dressed as a strength.

The role expansion also had a structural cause I should name honestly: Edward and Zian weren't self-directed. They did what they were asked to do—and that's fair—but they weren't driving things forward on their own initiative. That meant Robin and I absorbed the unassigned work by default, and I absorbed more of it than Robin because I had the laptop-based simulation path while Robin's work depended on hardware access we didn't have.

1b. The contribution I'm proudest of isn't the 820-plus commits or the retrained model. It's what happened on 2026-03-11, the first cancelled flight day. Weather killed our only window. Instead of going home I calibrated the focal length in the lab, ran lens calibration, wrote a quick-reference sheet for field use without internet, and closed ten commits before dark. That day crystallised my thesis: *the simulation framework was the only reason this project produced verifiable engineering output at all*. Given that we had one real flight video from Sid—one—I built the entire GPS estimation pipeline, calibration tools, and video analysis scripts around it. I squeezed everything I could out of extremely limited data, and I'm genuinely proud of that.

The BGR camera debugging (2026-02-17) is the incident I'd call reflection-in-action (Schön, 1983). I spent hours trying AWB modes, manual gains, ISP tuning—all wrong. Then I stopped guessing and systematically tested all six channel permutations. Root cause: the IMX296 outputs BGR despite labelling itself RGB888. The ratio—three hours of intuition before switching to systematic diagnosis—tells me something about how I respond to frustration. I reach for plausible explanations before exhaustive ones, and the switch happens only after frustration builds up enough to override the comfort of guessing.

1c. The distinction I'll focus on in future work is between *capability* and *enablement*. The evidence: the `-robin` flag and HTTP polling interface I built in wk20 let Robin integrate my vision output without touching my code. He worked for two days without asking a single question. That was the first teamwork move I made that scaled without my presence—and it happened in wk20, not wk12. In my next team project I'll schedule explicit pair-programming sessions from week one. Falsifiable signal: can a teammate

complete a handoff task independently within 48 hours? If not, my documentation has failed, not their attention.

My team

2a. At the wk12 kickoff we split on module ownership: Robin took flight execution, I took perception, Harish took hardware and comms, Edward shared CV with me, and Zian took path planning. We didn’t appoint a single technical lead or a formal project manager—the brief devolves those choices to the team, and nobody pushed for the role. That was a decision by omission, and it cost us later. Table 2 maps what actually happened against Tuckman’s (1965) stages.

Table 2: Team development mapped to Tuckman (1965) stages

Stage	When	Evidence	Gaps / what I’d change
Forming	wk12–14	Polite role division at one meeting. No interface contracts. Everyone looking for leadership nobody claimed.	[APOLLO: was there a specific moment where you felt leaderless? Any awkward silences?] Too brief—one meeting isn’t forming, it’s task allocation. I’d front-load a whiteboard session defining inputs/outputs at every module boundary.
Storming	wk15–18	Hardware failures killed confidence. Robin and I discovered overlapping state machines around wk17. Edward’s trained model couldn’t be validated without flight data. Team morale dropped after second cancelled flight day.	[APOLLO: did anyone voice frustration openly, or was it all private? Any specific argument or tension moment?] I proposed a complex FSM; Robin pushed back on maintainability. I should have surfaced that tension earlier instead of letting it build.
Norming	wk19–20 (Easter)	Hardware finally fixed during Easter. State machine confirmed working. I wrote handoff docs and the <code>-robin</code> integration flag; Robin integrated without questions for two days. Complaint letter drafted and iterated collaboratively.	[APOLLO: who suggested spending Easter working? Was it unanimous?] Norming came late—wk19 out of 23—because we couldn’t norm around hardware that didn’t exist.
Performing	wk21–22	Parallel tracks productive. Demo day produced a working system. Robin’s state machine consumed my CV output cleanly.	Performing lasted roughly 1.5 weeks. If we’d had working hardware from wk12, we could have reached this stage by wk16.

The most dangerous row is the gap between Forming and Storming. Our Forming lasted one meeting. In Lencioni’s (2002) terms, we divided tasks but never committed to a cadence for re-evaluating that division when circumstances changed. When the environment *did* change—three cancelled flight days shifted all progress from the hardware bench to my laptop—the structure drifted silently because we had no routine for revisiting it. Unexplicit structure is a luxury that only survives in slack-rich teams, and ours had no slack.

The hardware situation deserves honest mention. All other teams had working hardware from the start. Ours wasn’t functional until Easter. The first two practice flight days effectively didn’t count—we arrived with scripts ready to run, a simulation to validate, a progressive test ladder prepared, and the hardware just didn’t work. That’s three flight days lost to equipment we didn’t provide and couldn’t control. We wrote a complaint letter about it, iterated the tone six times across all five team members, and sent it to the course organiser. The letter wasn’t blame—it was a factual record of what we couldn’t test and why.

2b. The most impactful team-working element, measured by what it unblocked, was documentation that traded my time for Robin’s. The four hours I spent writing the vision pipeline handoff doc and the `-robin` integration flag bought Robin roughly twenty hours of unblocked integration in wk20. He wired my detection output into his mission script without asking a single clarifying question. I now understand that worked because it traded solo time for onboarding time at roughly 5:1—better team economics than I’d assumed documentation ever delivered.

The wk17 FSM overlap needs to be on record. I’d built a complex state machine with timeout guards and a geofence repulsor. Robin argued it was over-engineered for a team of five with limited testing time. My first reaction was defensive—the FSM was *my* code, and what felt like technical conviction was partly territorial reflex. Using Lencioni’s pyramid, I was ring-fencing work because I didn’t trust the team to navigate it. I took a day, reread Robin’s flight script, and proposed a compromise: keep the full FSM for autonomous mode, strip it to lightweight HTTP polling for Robin’s integration. He accepted. The integration took an afternoon instead of a week.

Robin and I were the two proactive members. Edward and Zian delivered assigned work competently but didn’t drive things forward independently. [APOLLO: is this fair? Would you soften it? Any specific example of Edward or Zian stepping up that you haven’t mentioned?] I don’t say this to assign blame—in a five-person team with unclear leadership, “do what you’re told” is a rational response. But it meant that unassigned work defaulted to two people, and that asymmetry shaped everything from workload to morale.

2c. Three changes, each falsifiable. First: a weekly one-line decision log from wk1—every call that binds more than one person’s work gets written down. The silent structural drift after wk16 would have been caught by any written record. Second: interface contracts *before* parallel work begins. A single wk14 session defining the HTTP contract between my vision output and Robin’s state machine would have prevented the wk17 overlap entirely. Third: stop treating documentation as a substitute for conversation. The two pieces of writing that actually changed behaviour were the short pivot message and the complaint letter—both brief, timely, paired with a live discussion. Any document over 500 words gets a 15-minute walkthrough within a week, or I treat it as unsent.

My impact

3a. I’ll state the thesis plainly: the simulation framework I built is the reason this project produced verifiable engineering output despite three effectively lost flight days. The mechanism matters more than the claim, so Table 3 maps what I actually did, who it helped, and what changed.

Table 3: Contribution–impact chain

What I built / did	Who it helped	Measurable outcome
CV pipeline: detection → GPS with pitch/roll correction, altitude compensation	Robin (state machine); whole team	Robin integrated via HTTP polling for 2 days unaided; worked on demo day
GPS averaging techniques, tested against single DJI video from Sid	Team (casualty position accuracy)	Single-frame estimates carry significant error; averaging reduced CEP; deduced ground truth from the video
Interactive simulator (2508 lines) with GPS drift, camera shake, blur	Whole team (8 weeks with no flight data)	R01–R12 verified in sim; sim-to-real delta was thin on demo day
Safety: geofence repulsor + zone enforcement (Robin suggested concept)	Robin; pilot	Edge-case tested over several days (dummy near NFZ boundary)
Test ladder (0a–4), 127 unit + 146 pytest tests	Team (confidence building)	Ready but never fully run on real hardware due to limited flight time
FOV/lens calibration; video analysis tools for DJI footage	Team (coordinate accuracy)	Maximised learning from one real video (all we had)
Dashboards: passive watch, diagnostics, ground station (headless Pi)	Pilot; Robin	Pilot operated 4-button UI one-handed while holding RC
Complaint letter (6 iterations, all 5 names)	Team	[APOLLO: did Steve respond? outcome?]

The Kolb (1984) cycle ran deliberately, not retrospectively. *Concrete experience*: three collapsed flight days — practice days 1 and 2 produced zero usable flight time because we were given faulty hardware, and all the other teams had working hardware from the start. *Reflective observation*: the team’s definition of progress assumed hardware access we didn’t have. Four of us were idle because planned work needed a drone in the air; I was blocked too, but I had a laptop. That asymmetry was the data point. *Abstract conceptualisation*: if `vision.py` exposed a single interface (`detect_in_image` → `(found,`

`x, y, conf`)) and auto-selected between Ultralytics on laptop and TFLite on Pi, then the same state machine could run in both places. Anything proven in simulation would, in principle, run on hardware. *Active experimentation*: six weeks building on that premise. When we finally ran on the Pi, the delta was thin — a bounding-box coordinate bug, a double-scaling fix, a deleted calibration file — and the system came up.

But the sober closing: the simulation couldn't tell us what it couldn't tell us. It couldn't predict motion blur at 5 m/s with a real IMX296 — I only modelled Gaussian shake, not directional smear. It couldn't find the descent stall I accepted as a SITL artefact at 4–5 m. And it couldn't reveal what GPS timing lag (100–200 ms latency, ≈ 1 m error at 5 m/s) would do to a multi-pass lock. Sim-to-real parity is an argument I can only win about the things I modelled.

Against R01–R12, our system hit 12/12 in simulation but only [X/12 — Apollo to confirm actual number] on real hardware by flight day. Some of those gaps were bench-test items we didn't schedule — coordination failure, not engineering failure. That distinction matters: I mistook technical delivery for team enablement.

3b. The metric I didn't track until too late: the number of times a teammate other than me edited `simple_simulator.py` or `main.py` was zero. That's the gap. Three mechanisms for future projects, each with a test:

1. Every new tool must have a named teammate beneficiary documented *before* it ships. If I can't name one, I don't start.
2. No file over 500 lines gets refactored solo. I pair on the review, and the signal is whether the partner can edit independently within 48 hours.
3. Two hours per week of explicit teaching, graded not by whether I explained but by whether the teammate completes a task within 48 hours without asking me a second question.

The simulation was the right call — it kept the project alive when hardware failed us. But the fact that it lived entirely in my head, and nobody else could maintain it, is a design flaw in how I work, not a strength. I want to resolve that tension in my [internship / next role — Apollo to specify].

My support

4a. A generic thank-you list caps at Merit. I'll name the specific mechanism — what each person did that changed how I worked.

Table 4: Support received — who, what, and how it changed my working

From whom	What they specifically did	How it changed my work
Robin	Refused to integrate against my first state machine (wk17). Argued it was over-engineered and unmaintainable	Forced the <code>main.py</code> refactor (1411→754 lines). Made me separate “right design” from “right design for this team.” Led to the HTTP interface he integrated against unaided
Robin	Suggested repulsive-force concept for NFZ avoidance	I ran with it — developed and tested it over several days. Became a core safety feature of the simulation
Sid	Provided the single DJI flight video with SRT telemetry	That one video was all the real data I had. Anchored FOV calibration (54.4°), validated GPS averaging, fed model retraining (mAP50 = 0.995). I squeezed everything possible from it
Robin	[APOLLO: you mentioned Robin cut sharp sentences from the complaint letter — confirm: which draft, what he said, how it changed the tone]	[How did that shape your approach to written confrontation?]
Harish	[APOLLO: Harish handled hardware + comms. Did he debug something that unblocked you? Set up the Cube wiring? Specific incident?]	[Effect on your work]
Edward	[APOLLO: Edward shared CV responsibility. Did he flag a technical issue? Help on a flight day? Did his trained model inform any of your decisions, even if you didn’t use it?]	[Effect on your work]
Zian	[APOLLO: Zian did path planning. Did his output feed into your simulator? Did he contribute anything that shaped your work?]	[Effect on your work]

Robin’s wk17 pushback is the episode that matters most. What I’d framed as a technical disagreement was actually a maintainability one, and I preferred the technical framing because it protected me from noticing I’d built something only I could read. In Lencioni’s (2002) terms, I was ring-fencing work because I didn’t trust the team to navigate it — and that looked like technical conviction when it was territorial reflex. I proposed a compromise: keep the full FSM for autonomous mode, strip it to HTTP polling for Robin’s script. He accepted. Integration took an afternoon instead of a week.

Robin and I were the two proactive members of the team. We worked in close coordination on the CV-to-state-machine integration, and ideas were discussed between us as colleagues — he’d suggest a concept like the repulsive force, I’d take it and develop it, neither of us treated it as a transaction. That dynamic was the closest thing to real engineering collaboration I experienced on the project.

[NOTE TO APOLLO: §4a is thin because you haven’t shared specific support from Harish, Edward, or Zian. The brief expects balanced reflection on what others did FOR you, not just what you did for them. Even small things count: equipment setup, a question that made you rethink, covering a task, moral support on a bad day. This is the biggest gap right now.]

4b. Table 5 maps what I did that helped others work more effectively.

Table 5: Support given — action, recipient, evidence it worked

For whom	What I did	Evidence of effect
Robin	Built <code>-robin</code> flag + HTTP polling so his state machine could consume vision output without touching my code	Integrated for 2 days without a single question. First teamwork move that scaled without my presence
Robin	Provided detection images + attitude-corrected GPS coordinates via defined interface	His state machine made decisions from my output. Building this was expensive (pitch/roll, altitude, averaging) but he never had to understand my internals
Pilot	Redesigned ground station from dense grid to 4-button set (Y/N/M/L) after pilot said original too dense for one-handed RC use	[APOLLO: confirm — did pilot use it on demo day? Specific evidence?]
Demetro	[APOLLO: you mentioned building Demetro’s FDR slides. Confirm: how many slides, how many iterations, what happened at FDR day]	[Outcome — did he deliver them? Behaviour change?]
Whole team	Documentation infrastructure: field quick-ref for no-internet use, handoff pipeline docs, blueprints	Team operated on flight day without needing me at every decision
Whole team	Drove the 6-iteration complaint letter about faulty provided hardware (all 5 names)	[APOLLO: was it sent? Did Steve respond? Outcome?]
Whole team	Simulation as morale anchor during failed flight days	Gave the team a concrete thing to point to when hardware kept failing and morale was lowest

The Robin handoff deserves scrutiny. The `passive_watch` HTTP interface traded my solo time for his onboarding time at roughly 5:1 — four hours writing docs and building the flag bought him twenty hours of unblocked work. I hadn’t believed documentation delivered that kind of return. It worked because it gave Robin everything he needed without requiring him to understand my internals. That’s the line between capability (I can build it) and enablement (someone else can use it without me) — and I didn’t cross it until wk20.

During the failed practice days, the whole team felt demotivated — me included. I had loads of scripts ready to test and nowhere to test them. Instead of just sitting with the frustration, I went deeper into the simulation, running it thousands of times, modelling camera blur, thinking through everything that could go wrong. The simulation gave the team something to show when hardware couldn’t, and it gave Robin’s state machine a testbed that didn’t require a drone in the air.

What I leave believing. I came as a Ukrainian-trained engineer whose reflex under failure was “one person delivers.” I leave believing its near-inversion: *the engineer who delivers alone is one recruitment away from being the single point of failure in whatever she builds*. The wk17 conflict and Robin’s wk20 unaided handoff told me, in different registers, that the same work which delivered the report was the work that kept my teammates at the edge of their own project. Falsifiable internship signal: in my first six weeks, does at least one teammate modify a module I own without asking permission? If yes, even once, I’ve started to become an engineer whose work enables other people’s work. That’s the only kind of engineering I now think is worth aspiring to, and I haven’t resolved it yet.